

Server & Client Components — Deep Dive

Beginner to Expert | Interview Q&A with Answers | RSC Internals

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

The Two Worlds — Server vs Client

Every component in the Next.js App Router lives in one of two worlds. **Server Components** (the default) run on Node.js and send zero JS to the browser. **Client Components** (opt-in with "use client") run in the browser and support interactivity.

■ SERVER COMPONENT

```
// No directive needed – default
async function UserCard({ id }) {
  // Runs on SERVER only
  const user = await db.user
    .findUnique({ where: { id } });
  return <div>{user.name}</div>;
}
```

CAN:

- async / await
- DB / Prisma direct access
- Secret env vars (process.env)
- File system (fs module)

CANNOT:

- useState / useEffect
- onClick / onChange
- window / document
- useRef (interactive)

■ CLIENT COMPONENT

```
// 'use client' at very top of file
'use client';
import { useState } from 'react';

function LikeButton({ count }) {
  const [n, setN] = useState(count);
  return (
    <button onClick={() => setN(n+1)}>
      Like {n}
    </button>
  );
}
```

CAN:

- useState / useEffect / useRef
- onClick / onChange events
- window / document / localStorage

CANNOT:

- async component function
- Direct DB / Prisma access
- Secret server env vars

The Golden Rule

■ Golden Rule: Default to Server Components. Add "use client" ONLY when you need: React hooks (useState/useEffect), event handlers (onClick/onChange), or browser APIs (window/document/localStorage). Push "use client" as DEEP (close to leaf components) as possible.

Full Comparison Table

Feature	Server Component	Client Component
---------	------------------	------------------

Directive	None (default)	'use client' at top of file
Runs on	Node.js server	Browser (+ server for initial HTML)
async/await	Yes	No — cannot be async
useState / useEffect	No	Yes
Event handlers	No	Yes (onClick, onChange...)
DB / Prisma access	Yes — direct	No — only via API routes
Secret env vars	Yes — safe	No — never put secrets here
Browser APIs	No	Yes (window, document, localStorage)
JS sent to browser	Zero bytes	Included in JS bundle
SEO	Excellent — pre-rendered	Good — hydrated after load
3rd party libraries	Must be server-compatible	All client libraries work

■ INTERMEDIATE LEVEL

Lifecycle — What Happens at Request Time

Step	Server Component	Client Component
1	Request hits Next.js server	Component bundled into JS at build
2	Async function runs on Node.js	Server pre-renders initial HTML
3	DB/API calls execute server-side	HTML sent to browser (fast first paint)
4	Renders to HTML + RSC Payload	Browser downloads JS bundle
5	HTML streamed to browser	React hydrates — attaches event listeners
6	ZERO JS sent for this component	Component fully interactive

The Composition Pattern — Mixing Server and Client

The real power comes from **composing** Server and Client Components together. Server Components handle data fetching. Client Components handle interactivity. Pass data from Server to Client as props.

```
// app/product/[id]/page.tsx — SERVER COMPONENT (default)
import { db } from '@lib/db';
import AddToCartButton from '@components/AddToCartButton';

async function ProductPage({ params }: { params: { id: string } }) {
  // Runs on server — direct Prisma access
  const product = await db.product.findUnique({ where: { id: params.id } });

  return (
    <div>
      <h1>{product?.name}</h1>          {/* Server-rendered — 0 JS */}
      <p>Price: Rs.{product?.price}</p>  {/* Server-rendered — 0 JS */}

      {/* Client Component nested inside Server Component */}
      <AddToCartButton productId={params.id} />
    </div>
  );
}

// components/AddToCartButton.tsx — CLIENT COMPONENT
'use client';
import { useState } from 'react';

export default function AddToCartButton({ productId }: { productId: string }) {
  const [added, setAdded] = useState(false);
  const handleAdd = async () => {
    await fetch('/api/cart', { method: 'POST', body: JSON.stringify({ productId }) });
    setAdded(true);
  };
  return (
    <button onClick={handleAdd}>
      {added ? 'Added to Cart!' : 'Add to Cart'}
    </button>
  );
}
```

The "use client" Boundary — How It Propagates

"use client" is a **module-level directive**. Once declared in a file, **every component imported below that file** also becomes a Client Component, even without their own directive. This is the most common source of accidental over-clientization in Next.js apps.

```
// Boundary propagation diagram:
app/page.tsx                (Server — no directive)
  ■■■ Dashboard.tsx         (Server — no directive)
    ■■■ Sidebar.tsx ('use client') ← BOUNDARY CREATED HERE
      ■■■ NavItem.tsx      (Client — NO directive but inside boundary)
      ■■■ UserAvatar.tsx   (Client — NO directive but inside boundary)

// Components ABOVE boundary = Server Components
// Components BELOW boundary = Client Components (even without directive)
```

```

// ■ WRONG — 'use client' too high (entire app becomes Client)
'use client';
// app/layout.tsx — This makes EVERYTHING a Client Component!
export default function RootLayout({ children }) { ... }

// ■ RIGHT — push 'use client' to the leaf component
// components/ThemeToggle.tsx (only this one is Client)
'use client';
import { useState } from 'react';
export default function ThemeToggle() {
  const [dark, setDark] = useState(false);
  return <button onClick={() => setDark(!dark)}>Toggle</button>;
}

// app/layout.tsx (stays as Server Component)
import ThemeToggle from '@components/ThemeToggle';
export default function RootLayout({ children }) {
  return (
    <html><body>
      <ThemeToggle /> { /* Only ThemeToggle is Client */}
      {children}
    </body></html>
  );
}

```

The "children" Pattern — Server Component Inside Client

You cannot **import** a Server Component inside a Client Component (it would cross the boundary and become Client). But you CAN pass it as **children** — this is composed ABOVE the boundary so it stays server-side.

```
// ■ WRONG — importing Server Component inside Client
'use client';
import ServerComponent from './ServerComponent'; // Now it's a Client Component!

// ■ RIGHT — compose in Server, pass as children
// app/page.tsx (SERVER Component)
import ClientWrapper from '@components/ClientWrapper';
import ServerContent from '@components/ServerContent';

export default function Page() {
  return (
    <ClientWrapper>
      <ServerContent /> { /* Stays as Server Component! */ }
    </ClientWrapper>
  );
}

// components/ClientWrapper.tsx
'use client';
import { useState } from 'react';

export default function ClientWrapper({ children }: { children: React.ReactNode }) {
  const [open, setOpen] = useState(true);
  return (
    <div>
      <button onClick={() => setOpen(!open)}>Toggle</button>
      {open && children} { /* children = ServerContent — still Server! */ }
    </div>
  );
}
```

Passing Data Server to Client — Serializable Props Only

Data Type	Serializable?	Example
string	Yes	"wheat-hybrid"
number	Yes	42, 3.14
boolean	Yes	true / false
array	Yes	[1, 2, 3]
plain object	Yes	{ id: "1", name: "Crop" }
null / undefined	Yes	null
function	NO	() => {} — use Server Actions instead
Date object	NO	Use date.toISOString() → string
Map / Set	NO	Convert to array/object first
class instance	NO	Convert to plain object first

How RSC Works Internally — Traditional SSR vs RSC

```
// Traditional SSR (Pages Router):
Server: React.renderToString(tree) → Full HTML string
Client: Downloads ALL component JS → Hydrates entire tree
Problem: 100% of components hydrated even if 90% are static

// RSC (App Router):
Server: Renders Server Components → RSC Payload (binary tree)
Client: Receives HTML (for display) + RSC Payload (for React)
React: Only Client Components get hydrated — Server = 0 bytes JS

// RSC Payload (simplified):
{
  type: "div",
  children: [
    { type: "h1", children: "Wheat Hybrid 50" }, // Server — inlined
    { type: "$CLIENT_REF:0", props: { id: "abc" } } // Client — ref
  ]
}
//                                     ↑ Browser downloads JS for this only
```

server-only and client-only Packages

These packages act as **compile-time guards** to prevent accidental cross-boundary imports. Import them at the top of sensitive files.

```
// lib/db.ts — server-only guard
import 'server-only'; // Build error if imported in Client Component
import { PrismaClient } from '@prisma/client';

const db = new PrismaClient();

export async function getUser(id: string) {
  return db.user.findUnique({ where: { id } });
}
// If a developer imports this in a Client Component → BUILD FAILS loudly
// Prevents accidental DB credential leakage

// lib/analytics.ts — client-only guard
import 'client-only'; // Build error if imported in Server Component

export function trackEvent(name: string) {
  window.gtag("event", name); // window — browser only
}
```

Context in Server Components — What Is and Isn't Allowed

```
// ■ Context does NOT work in Server Components
// import { createContext } from 'react'; // Won't work

// ■ Context only in Client Component files
'use client';
import { createContext, useContext, useState } from 'react';

type Theme = 'light' | 'dark';
const ThemeCtx = createContext<Theme>("light");

export function ThemeProvider({ children }: { children: React.ReactNode }) {
  const [theme, setTheme] = useState<Theme>("light");
  return (
    <ThemeCtx.Provider value={theme}>{children}</ThemeCtx.Provider>
  );
}
export const useTheme = () => useContext(ThemeCtx);

// app/layout.tsx (Server Component) – wraps with provider
import { ThemeProvider } from '@components/ThemeProvider';
export default function RootLayout({ children }) {
  return (
    <html><body>
      <ThemeProvider> { /* Client Provider wraps everything */}
        {children}
      </ThemeProvider>
    </body></html>
  );
}
```

Third-Party Library Wrapper Pattern

Many npm packages use hooks internally but don't declare "use client". Using them directly in Server Components causes errors. Always wrap them in your own Client Component.

```
// ■ Problem: react-hot-toast uses hooks internally
// import { Toaster } from 'react-hot-toast'; // Fails in Server Component

// ■ Solution: Wrap in your own Client Component
// components/ToastProvider.tsx
'use client';
import { Toaster } from 'react-hot-toast';

export default function ToastProvider() {
  return <Toaster position="top-right" />;
}

// app/layout.tsx (Server Component – safe now)
import ToastProvider from '@components/ToastProvider';
export default function RootLayout({ children }) {
  return (
    <html><body>
      <ToastProvider />
      {children}
    </body></html>
  );
}

// Common packages needing this pattern:
// react-hot-toast, framer-motion, react-query providers,
// recharts, react-dnd, some UI component libraries
```

■ Critical Gotchas

■ Gotcha 1: "use client" is a FILE-level boundary, not component-level. Every export in that file becomes a Client Component — even utility functions or sub-components you define in the same file.

■ Gotcha 2: You cannot pass functions as props from Server to Client Components — functions are not serializable. Define event handlers inside the Client Component. Use Server Actions for server mutations instead.

■ Gotcha 3: Passing a Date object from Server to Client throws a serialization error. Always call .toISOString() and pass the string. Same applies to Map, Set, and class instances — convert to plain objects first.

■ Gotcha 4: Using useState in a Server Component throws: "You're importing a component that needs useState. This only works in a Client Component." The fix: add "use client" or extract to a separate Client Component.

■ Best Practice: Keep layout.tsx, page.tsx, and data-fetching components as Server Components. Extract only interactive leaf elements (buttons, forms, modals) as Client Components. This minimizes JS bundle size.

■ Best Practice: Use the server-only package on files with DB access or secrets. It adds a safety net that causes a build error if someone accidentally imports them in a Client Component.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What is a Server Component in Next.js? What can it do?

Answer: A Server Component is the default component type in the App Router — no directive needed. It runs exclusively on the Node.js server, can be async, can directly access databases with Prisma, read secret env variables, and sends ZERO JavaScript to the browser. It cannot use React hooks, event handlers, or browser APIs.

BEGINNER

Q2. What is a Client Component and when should you use it?

Answer: A Client Component is declared with "use client" at the very top of the file. It runs in the browser and supports React hooks (useState, useEffect), event handlers (onClick, onChange), and browser APIs (window, document, localStorage). Use it only when you need interactivity. For everything else — data fetching, display, layouts — stay with Server Components to minimize your JS bundle.

BEGINNER

Q3. What are the key differences between Server and Client Components?

Answer: Server: default, async, DB/secret access, zero client JS, no hooks/events. Client: "use client" directive, hooks, events, browser APIs, included in JS bundle. Main rule: default to Server, add "use client" only when interactivity is needed. Push "use client" as deep (leaf level) as possible.

Intermediate Level

INTERMEDIATE

Q4. How does "use client" propagate in a component tree?

Answer: "use client" is a module-level boundary. Once declared, everything imported BELOW that file in the import chain also becomes a Client Component — even without their own directive. Components above the boundary remain Server Components. This is why placing "use client" on a layout.tsx accidentally makes the entire app a Client Component.

INTERMEDIATE

Q5. Can you use a Server Component inside a Client Component?

Answer: Not via import — importing a Server Component inside a Client file crosses the boundary and converts it to a Client Component. The correct pattern is the "children" prop: compose the Server Component as children in a Server Component above the boundary, then pass it as children prop into the Client Component. The children are evaluated above the boundary and stay server-side.

INTERMEDIATE

Q6. What data types can be passed as props from Server to Client Components?

Answer: Only serializable types: string, number, boolean, null, undefined, plain arrays, plain objects. NOT serializable: functions (use Server Actions instead), Date (use .toISOString()), Map, Set, class instances (convert to plain objects). Passing non-serializable props throws a Next.js error at runtime.

INTERMEDIATE**Q7. How do you use a third-party library that uses hooks in a Server Component context?**

Answer: Wrap it in your own Client Component file with "use client". The wrapper re-exports the library component with the client boundary. Then import your wrapper in Server Components — it is safe because the boundary is declared in your wrapper file. Common examples: react-hot-toast, framer-motion, recharts, UI component provider wrappers.

INTERMEDIATE**Q8. How do you set up React Context to work with Server Components?**

Answer: Context and createContext only work in Client Components. Create a Provider component in a "use client" file. In the Root Layout (Server Component), import and render the Provider — this is safe because you are composing it from the server side. Child Server Components inside the Provider still stay as Server Components. Client Components inside consume context via useContext as normal.

Expert / Syngenta-Style Questions

EXPERT**Q9. How does RSC differ from traditional SSR? What is RSC Payload?**

Answer: Traditional SSR (Pages Router) runs renderToString() and sends full HTML — then hydrates the ENTIRE React tree client-side. RSC generates a serialized React tree (RSC Payload) where Server Components are inlined as plain element descriptors (zero JS) and Client Components are referenced by ID (their JS downloaded separately). Only Client Components are hydrated. Server Components produce literally zero bytes of client JavaScript.

EXPERT**Q10. What is the server-only package? When and why would you use it?**

Answer: server-only is a tiny npm package — importing it in a file makes that file throw a BUILD ERROR if it is ever imported by a Client Component. Use it on files with Prisma queries, secret API keys, or any server-exclusive code. It prevents silent data leakage — instead of secrets accidentally reaching the browser bundle, the build fails loudly. Always add it to DB utility files in production apps.

SYNGENTA**Q11. Design the Server/Client Component split for a Syngenta crop data dashboard.**

Answer: Server Components: CropDashboardPage (fetches from Prisma, zero JS), CropDataTable (renders rows — pure display), WeatherSection (server fetches weather API). Client Components: FilterBar (useState for filter selections), CropPriceChart (recharts with useEffect), AlertBell (manages notification state), ExportButton (handles file download). Result: 80% of the dashboard is Server Components — minimal JS bundle. Only interactive elements ship JavaScript to the browser.

EXPERT

Q12. What happens when you add `useState` inside a Server Component? How do you fix it?

Answer: Next.js throws a build error: "You are importing a component that needs `useState`. This only works in a Client Component but none of its parents are marked with `use client`." Two fixes: (1) Add "`use client`" to that file if the whole component needs interactivity. (2) Extract just the stateful part into a separate Client Component file and import it — this keeps the parent as a Server Component and minimizes bundle size.